

APPLICATION ÉVÉNEMENTIELLE
HAHITANTSOA / TITAN

Guide complet de développement logiciel

VS Code + Codex | Django + React + PostgreSQL | Production-ready

Objet du document

Fournir une feuille de route exécutable, des conventions d'ingénierie et des prompts Codex prêts à l'emploi pour construire l'application événementielle à partir du CDC consolidé v3.2.

Champ	Valeur
Projet	ERP métier événementiel Hahitantsoa / Titan
Référence fonctionnelle	CDC fonctionnel et technique v3.2
Audience	Tech Lead, développeur full-stack, chef de projet, équipe de recette
Architecture cible	Monorepo - Django REST - React TypeScript - PostgreSQL - Celery - Redis
Version	1.0
Date	26 mai 2026
Statut	Guide de démarrage et d'implémentation

Document de travail professionnel - à utiliser avec le CDC v3.2 validé.

Note stratégique : comment démarrer le développement

Recommandation principale

Créer une nouvelle conversation dédiée au développement, idéalement dans le même Projet ChatGPT, puis utiliser le repository Git, AGENTS.md, les ADR et le CDC v3.2 comme sources de vérité. La présente conversation reste utile pour le cadrage métier et les évolutions client.

Cette recommandation évite de mélanger les versions précédentes du besoin, les corrections client et les décisions d'implémentation. Le projet a déjà évolué plusieurs fois : accès Internet, mobile/tablette, caisse, consommables, fournisseurs, documents numérisés, bon de sortie et échéances de paiement.

Conversation	Usage recommandé	Documents à joindre
Conversation actuelle	Historique, validation métier, nouvelles demandes client	Documents B annotés et CDC consolidés
Nouvelle conversation - Développement	Architecture, backlog technique, Codex, code review et implémentation	CDC v3.2 + present guide + ADR/AGENTS.md des qu'ils existent
Conversations ultérieures	Un epic majeur ou une revue spécifique, si nécessaire	Fichiers issus du repository, pas seulement l'historique de chat

Fondement : les Projets ChatGPT permettent de regrouper fichiers, instructions et conversations autour d'un effort de longue durée ; Codex lit AGENTS.md avant de travailler et peut ainsi appliquer les règles versionnées du repository de façon reproductible. [R1] [R2]

Sommaire

Section	Contenu
0	Référentiel fonctionnel et décisions préalables
1	Architecture cible recommandée
2	Setup environnement : VS Code, Codex et Docker
3	Architecture du repository
4	Conventions de code, linting et qualité
5	Workflow Git et CI/CD minimal
6	Workflow Codex et prompts par phase
7	Patterns backend
8	Patterns frontend
9	Stratégie de tests
10	Documentation automatique et gouvernance
11	Checklist avant production
12	Ordre concret d'exécution
13	Organisation des conversations et du contexte
Annexes	Premier prompt Codex et références

0. Référentiel fonctionnel et décisions préalables

0.1 Socle fonctionnel

Le développement est fondé sur le CDC v3.2 consolidé. Les demandes client à intégrer au codebase incluent : accès en réseau local et via Internet, utilisation tablette/mobile, stocks et consommables, fournisseurs et bons de commande, caisse multi-utilisateurs, comptage de billets, achats avec justificatifs numérisés, agenda visiteurs/prospects, affichage des matériels en location, numérisation des CIN/NIF/STAT, MVola, bon de sortie interne et bon de livraison client.

0.2 Point à faire valider : Titan et les locaux

Ambiguïté métier à lever

Le document client v3.1 indique simultanément que Titan concerne la location de matériel sans local et liste les locaux parmi les offres Titan. La solution technique doit donc rester configurable jusqu'à validation définitive.

Sujet	Décision de conception provisoire
Hahitantsoa	Événement complet : local, articles, services et prestations annexes.
Titan	Location pure sans services annexes.
Local dans Titan	Option paramétrée : <code>allow_venue_in_titan</code> = true/false.
Règle de livraison	Bon de sortie interne pour préparation ; bon de livraison pour passation client.

0.3 Invariants métier

Code	Invariant
INV-001	Un proforma est une estimation ; il ne confirme pas définitivement la réservation.
INV-002	La réservation devient confirmée uniquement avec contrat signé et acompte reçu.
INV-003	Les disponibilités sont recontrôlées atomiquement au moment de la confirmation.
INV-004	Toute modification après contrat signé passe par proforma de modification puis avenant.
INV-005	Échéances initiales : 50 % du reste à payer à J-30, puis solde final et caution à J-10.
INV-006	Les ressources physiques sont partagées entre Hahitantsoa et Titan.
INV-007	Les documents sensibles ne sont jamais exposés publiquement.
INV-008	Toute action sensible produit une trace d'audit.

1. Architecture cible recommandée

1.1 Décision d'architecture

Architecture retenue : monorepo, monolithe modulaire Django, frontend React distinct, base PostgreSQL, traitements asynchrones Celery et notifications temps réel. Cette architecture privilégie la cohérence transactionnelle des processus réservation-stock-paiement-caisse avant toute décomposition en microservices.

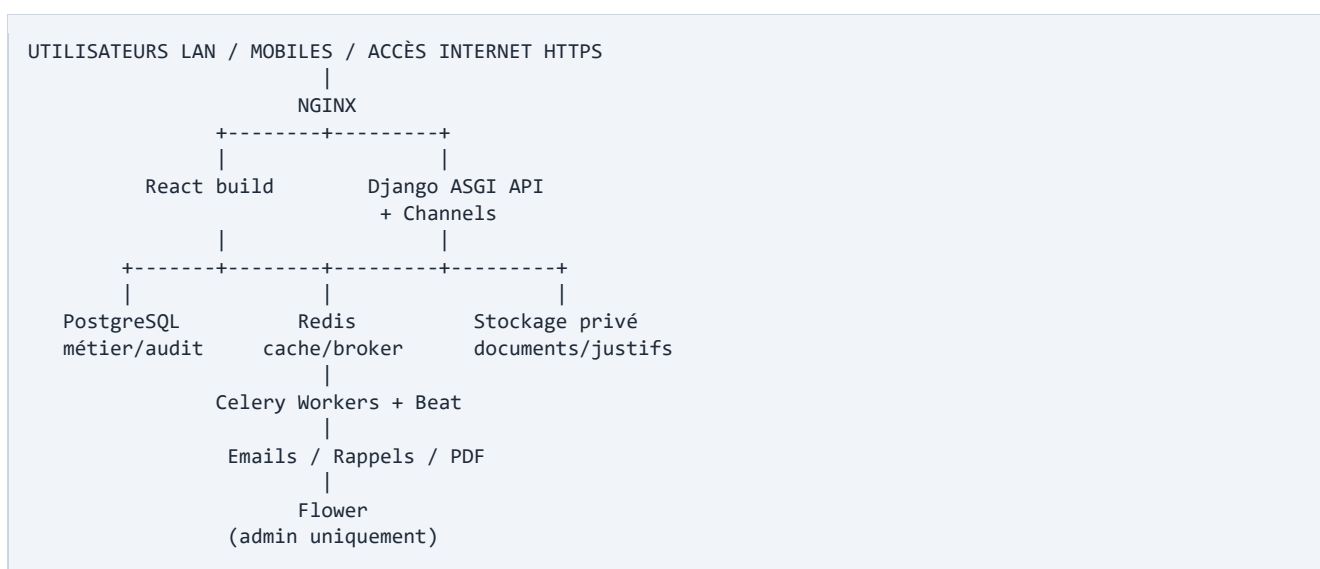
1.2 Stack retenue

Couche	Choix	Justification
Backend	Python 3.13 + Django 5.2 LTS	Framework mature ; LTS ; logique métier et authentification.
API	Django REST Framework + drf-spectacular	API versionnée et schéma OpenAPI.
Frontend	React + TypeScript + Vite	Interfaces riches, responsives et typées.
Données	PostgreSQL	Transactions, contraintes, verrous de lignes.

Couche	Choix	Justification
Async	Celery + Redis + Beat	Rappels, PDF, emails, échéances.
Temps reel	Django Channels + Redis	Notifications et actualisation calendrier.
Serving	Nginx + serveur ASGI	HTTP, WebSockets et publication HTTPS.
Déploiement	Docker Compose	Services reproductibles en local et production initiale.
Documentation	MkDocs + ADR + OpenAPI	Documentation versionnée.
Tests	pytest, Vitest, Playwright	Couverture unitaire, intégration et parcours.

Django 5.2 est une version LTS et prend en charge Python 3.13 ; PostgreSQL et les transactions/verrous répondent aux risques de double allocation ; Celery Beat prend en charge les tâches périodiques ; Docker Compose décrit et démarre les services interdépendants. [R3] [R4] [R5] [R6]

1.3 Schéma logique



1.4 Exposition Internet

Élément	Politique
Application	Accessible en HTTPS via le domaine configuré.
PostgreSQL / Redis	Non exposés directement à Internet.
Flower	Réservé au LAN, VPN ou administrateur.
Documents CIN/NIF/STAT	Stockage privé et téléchargement après autorisation backend.
Secrets	Variables protégées, hors repository.
Production	Exécuter <code>manage.py check --deploy</code> avant mise en ligne.

La checklist officielle Django de déploiement impose notamment de protéger les secrets, de configurer correctement l'environnement de production et de vérifier le projet avant publication. [R7]

2. Setup environnement : VS Code, Codex et Docker

2.1 Poste de développement

Composant	Choix recommandé	Usage
Système	Windows 10/11 + WSL2 Ubuntu	Environnement Linux cohérent avec le serveur.
Éditeur	VS Code connecté à WSL2	Édition, terminal et extensions.
Conteneurs	Docker Desktop + intégration WSL2	PostgreSQL, Redis et services applicatifs.
Agent IA	Codex IDE extension / CLI	Implémentation assistée et review.
Versionnement	Git + GitHub	Checkpoints, branches et Pull Requests.

L'extension Codex IDE s'utilise directement dans VS Code et Codex peut lire, modifier et exécuter le code. La documentation officielle indique également l'utilisation possible de WSL2 sous Windows. [R8] [R9]

2.2 Extensions VS Code

Extension	Utilité
OpenAI Codex	Implémentation assistée et review.
Python + Pylance	Support Django et analyse Python.
Ruff + Black Formatter	Linting et formatage backend.
ESLint + Prettier	Qualité frontend.
Docker	Gestion Compose et conteneurs.
GitLens	Historique et review Git.
Playwright Test	Exécution des parcours end-to-end.
REST Client / Thunder Client	Vérification API.

2.3 Initialisation du workspace

Commandes de démarrage - terminal WSL2

```
mkdir -p ~/projects/hahitantsoa-titan-erp
cd ~/projects/hahitantsoa-titan-erp
git init
git branch -M main

mkdir backend frontend infra docs scripts tests
mkdir -p docs/adr docs/architecture docs/runbooks docs/business-rules
mkdir -p tests/e2e .github/workflows
touch README.md AGENTS.md PLANS.md .editorconfig .gitignore .env.example
```

2.4 Docker dès le début

Décision

Dockeriser dès la phase Foundation. PostgreSQL, Redis et les comportements de concurrence font partie du cœur du système ; les tester tardivement exposerait le projet à des reworks coûteux.

Service	Phase Foundation	Production
db - PostgreSQL	Oui	Oui
redis	Oui	Oui
backend - Django	Oui	Oui
frontend - React/Vite	Oui	Build servi par Nginx
worker / beat / flower	Phase Notifications	Oui
nginx	Phase Intégration	Oui

3. Architecture du repository

3.1 Monorepo recommandé

Critère	Monorepo	Décision
Évolutions frontend/backend conjointes	Une seule PR et une seule version	Favorable
OpenAPI et client TypeScript	Génération centralisée	Favorable
Documentation et ADR	Un seul référentiel	Favorable
Contexte Codex	Instructions et code réunis	Favorable
Déploiement Compose	Orchestration simplifiée	Favorable

3.2 Arborescence cible

```

hahitantsoa-titan-erp/
|-- AGENTS.md                # Règles permanentes pour Codex
|-- PLANS.md                 # Plan d'exécution validé
|-- README.md
|-- compose.yaml
|-- compose.prod.yaml
|-- backend/
|   |-- config/settings/     # base, local, test, production
|   |-- apps/
|       |-- identity/        # auth, rôles, permissions
|       |-- crm/              # clients et documents
|       |-- appointments/    # visiteurs / prospects
|       |-- catalog/         # lieux, articles, services, packs
|       |-- inventory/       # stock et consommables
|       |-- suppliers/       # fournisseurs
|       |-- procurement/     # achats / bons fournisseurs
|       |-- bookings/        # réservations / calendrier
|       |-- contracts/       # contrats / avenants
|       |-- billing/         # proformas / factures / reçus
|       |-- payments/        # paiements / échéances
|       |-- cashbox/         # caisses utilisateurs
|       |-- logistics/       # sortie / livraison / retour / casse
|       |-- notifications/   # emails / signaux / scheduler
|       |-- audit/           # journal actions
|-- frontend/src/
|   |-- app/                 # router, providers, auth, layouts
|   |-- features/            # modules par fonctionnalité métier
|   |-- shared/              # UI, api, hooks, permissions
|-- docs/
|   |-- adr/
|   |-- architecture/
|   |-- business-rules/
|   |-- runbooks/
|-- tests/e2e/
|-- infra/nginx/
|-- .github/workflows/

```

3.3 Domaines backend

Domaine	Responsabilité
identity	Utilisateurs, rôles, sessions, permissions.
crm / files	Clients, sociétés, CIN/NIF/STAT et fichiers privés.
catalog / inventory	Catalogue, stocks physiques, consommables, disponibilités.
bookings	Réservations, calendrier, allocations partagées.
contracts / billing / payments	Proformas, contrats, factures, reçus, RAP, caution.
cashbox / procurement	Caisse, billets, achats, fournisseurs.
logistics	Bons, sorties, retours, casses.
notifications / audit	Rappels, temps réel et traçabilité.

4. Conventions de code, linting et qualité

4.1 Règle générale

Le code généré ou modifié avec Codex suit exactement les mêmes quality gates qu'un code écrit manuellement : formatage, linting, typage, tests, documentation, revue humaine et vérification des migrations.

4.2 Backend Python

Outil	Rôle	Décision
Black	Formatage Python	Formateur officiel du repository.
Ruff	Linting, imports et corrections simples	Pas de double formatage automatique avec Black.
mypy	Vérification statique des types	Mode strict avec exceptions ciblées.
pytest / pytest-django	Tests métier et API	Obligatoire avant merge.

Extrait cible - backend/pyproject.toml

```
[tool.black]
line-length = 100
target-version = ["py313"]

[tool.ruff]
line-length = 100
target-version = "py313"
exclude = ["migrations", ".venv", "generated"]

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "SIM", "DJ", "RUF"]

[tool.mypy]
python_version = "3.13"
strict = true
plugins = ["mypy_django_plugin.main"]
```

4.3 Frontend TypeScript

Outil	Usage
TypeScript strict	Contrats de données et prevention d'erreurs.
ESLint flat config	Règles de code et React hooks.
Prettier	Formatage uniquement.
Vitest + Testing Library	Tests de composants et hooks.

4.4 Conventions de nommage

Élément	Convention	Exemple
App Django	snake_case métier	cashbox, bookings

Élément	Convention	Exemple
Modèle Django	PascalCase	Reservation, CashSession
Service commande	verbe d'action	confirm_reservation()
Selector lecture	get/list explicite	get_available_resources()
Route API	pluriel kebab-case	/api/v1/delivery-notes/
Composant React	PascalCase	ReservationCalendar.tsx
Hook React	use + action	useRegisterPayment()
ADR	numéro + décision	ADR-004-reservation-confirmation.md

5. Workflow Git et CI/CD minimal

5.1 Stratégie Git

Stratégie recommandée : trunk-based development avec branches courtes et Pull Request obligatoire.

```
main
|-- feat/identity-authentication
|-- feat/client-company-records
|-- feat/reservation-confirmation
|-- feat/cashbox-closing
|-- fix/stock-concurrency
|-- docs/adr-titan-policy
```

Règle	Application
main	Toujours déployable.
Branches	Une fonctionnalité ou correction par branche courte.
Pull Request	Review humaine obligatoire et review Codex complémentaire.
Commits	Conventional Commits recommandé.
Migrations	Versionnées et relues dans chaque PR.
Tests	Tous les contrôles pertinents doivent réussir avant merge.

5.2 CI minimale

Pipeline	Contrôles obligatoires
Backend	Black check, Ruff, mypy, migrations check, pytest, Django deploy check.
Frontend	ESLint, Prettier check, typecheck, tests unitaires, build.
E2E	Docker Compose, données de test, Playwright, rapport artifacts.
Docker	Build reproductible des images et contrôle de configuration.

5.3 Review Codex

Codex peut être utilisé pour analyser une Pull Request et appliquer les consignes du fichier AGENTS.md. La documentation officielle recommande également des checkpoints Git avant et après les tâches modifiant le codebase. [R2] [R10]

6. Workflow Codex et prompts par phase

6.1 Principe de prompting

Ne jamais demander à Codex de 'développer toute l'application'. Chaque mission doit préciser : contexte, règles métier, fichiers autorisés, critères d'acceptation, tests attendus et résultat de review. AGENTS.md devient le cadre permanent ; chaque prompt traite une unité de travail vérifiable.

6.2 Contenu minimal de AGENTS.md

```
# AGENTS.md - Hahitantsoa / Titan ERP

## Domain invariants
- A proforma never confirms a reservation.
- A reservation is confirmed only after signed contract and received deposit.
- Availability must be rechecked atomically at confirmation time.
- Signed contracts cannot be modified directly; use amendments.
- Payment milestones: 50% of remaining balance at D-30, balance plus caution at D-10.
- Internal preparation uses a Bon de sortie; client handover uses a Bon de livraison.
- Stock is shared across Hahitantsoa and Titan.
- Identity documents and payment evidence are private.

## Engineering rules
- Thin API views and serializers.
- Write-side business rules in services; reads in selectors.
- transaction.atomic() and select_for_update() for critical allocations and numbering.
- Audit every sensitive write.
- Add tests and update OpenAPI/docs for changed behavior.

## Definition of done
- Run lint, typing and tests for modified layers.
- Summarize files changed, commands run, results and remaining risks.
```

Codex lit les fichiers AGENTS.md avant son travail et permet des instructions hiérarchisées par repository ou sous-dossier. [R2]

Phase 0 - Cadrage technique et ADR

Prompt Codex prêt à copier

```
Tu travailles sur le projet Hahitantsoa/Titan ERP.
Avant tout code :
1. Lis AGENTS.md, README.md et docs/.
2. Crée PLANS.md.
3. Rédige les ADR : monorepo ; monolithe modulaire ; stack Django/React/PostgreSQL/Celery ;
   confirmation après contrat + acompte ; politique Titan/local configurable ; stockage privé des pieces.
4. Crée docs/business-rules/reservations.md, billing.md, cashbox.md et logistics.md.
5. Liste les points client à confirmer.
6. Ne génère aucun modèle Django dans cette tâche.
Termine par une auto-review de cohérence et la liste des fichiers créés.
```

Phase 1 - Foundation

Prompt Codex prêt à copier

Implémente la fondation technique selon AGENTS.md et PLANS.md.

- Django 5.2 LTS avec settings base/local/test/production.
- PostgreSQL et Redis via Docker Compose.
- React + TypeScript + Vite.
- Endpoint GET /api/v1/health/.
- Black, Ruff, mypy, pytest ; ESLint, Prettier et TypeScript strict.
- Makefile et CI backend/frontend.
- Aucun modèle métier.

Affiche d'abord le plan ; exécute les checks ; justifie chaque dépendance ajoutée.

Phase 2 - Identity et permissions

Prompt Codex prêt à copier

Implémente identity et l'intégration frontend minimale.

- User personnalisé dès la première migration métier.
- Rôles : ADMIN, MANAGER, FRONT_DESK, STOREKEEPER, ACCOUNTANT.
- Sessions Django sécurisées.
- Vérification obligatoire des permissions côté backend.
- AuditLog sur création/modification/désactivation utilisateur.

API : login, logout, me et CRUD utilisateurs réservé ADMIN.
Frontend : LoginPage, ProtectedRoute, PermissionGate et layout responsive.
Ajoute tests permissions et exécute les quality gates.

Phase 3 - Référentiels

Prompt Codex prêt à copier

Implémente CRM, agenda, catalogue, fournisseurs et fichiers privés.

- Client particulier ou société ; documents CIN, NIF, STAT, RCS.
- Documents stocks par un service privé, sans URL publique directe.
- Rendez-vous visiteurs convertibles en client.
- Hahitantsoa : lieux, articles et services.
- Titan : location pure ; autorisation des lieux via BusinessScopePolicy.
- Salle Hahitantsoa par défaut configurable et modifiable par dossier.

Produis API DRF, OpenAPI et tests de permission/validation/fichiers.

Phase 4 - Réservation et disponibilité

Prompt Codex prêt à copier

Implémente bookings avec calendrier et confirmation transactionnelle.

- Le proforma ne crée aucune allocation active.
- Confirmation uniquement si contrat signé et acompte reçu.
- Dans confirm_reservation(), utiliser transaction.atomic() et select_for_update().
- Disponibilités partagées Hahitantsoa/Titan.
- API calendrier : disponibles, confirmés/en location, rendez-vous visiteurs.
- Tests de concurrence sous PostgreSQL.

Avant implémentation, propose les modèles, contraintes, endpoints et index.

Phase 5 - Documents et paiements

Prompt Codex prêt à copier

Implémente contracts, billing et payments.

- Proforma depuis réservation ; contrat depuis proforma accepté.
- Contrat signé immutable ; changement via proforma de modification + avenant.
- Paiements : CASH, MVOLA, CHEQUE, BANK_TRANSFER.
- Un reçu numéroté par paiement validé.
- Acompte nécessaire à confirmation.
- Échéances J-30 et J-10 ; caution au solde.
- Numérotation sans doublon sous transaction.
- Audit et événement outbox sur écritures sensibles.

Ajoute tests immutabilité, échéances, reçus, concurrence et permissions.

Phase 6 - Caisse et approvisionnement

Prompt Codex prêt à copier

Implémente cashbox, procurement, suppliers et consommables.

- Caisse par session utilisateur : ouverture, mouvements, clôture.
- Paiement CASH exige une session ouverte.
- MVOLA/cheque/virement n'alimentent pas le comptage de billets.
- Calcul du total physique par coupures et justification obligatoire des écarts.
- Achats/depenses avec justificatif privé.
- Seuil consommable déclenche une alerte et brouillon de bon fournisseur.
- Exports XLSX/PDF protégés par permission.

Ajoute les tests critiques associés.

Phase 7 - Logistique

Prompt Codex prêt à copier

Implémente logistics.

Workflow : bon de sortie interne -> sortie validée -> en location -> bon de livraison client -> retour -> contrôle intact/casse/manquant -> bon de retour -> mise à jour stock -> facture de casse et application caution si nécessaire.

Contraintes : quantités sorties <= confirmées ; mouvements immuables ; audit ; PDF imprimables.

Teste parcours nominal, casse, manque et sortie concurrente.

Phase 8 - Notifications et temps réel

Prompt Codex prêt à copier

Implémente notifications, Celery Beat et Django Channels.

- Rappels J-30, J-10, retours attendus et seuils consommables.
- Barre de notifications et actualisation calendrier/stock.
- Utiliser transaction.on_commit() ou outbox après transaction validée.
- Tâches idempotentes.
- Flower non exposé publiquement.

Teste l'idempotence et autorisations WebSocket.

Phase 9 - Frontend complet

Prompt Codex prêt à copier

```
Implémente les écrans React du MVP à partir d'OpenAPI.
- Feature-based architecture ; TypeScript strict ; TanStack Query ; React Router.
- Responsive desktop/tablette/mobile.
- Clients, agenda, catalogue, calendrier, documents, paiements, caisse,
  consommables, fournisseurs, logistique et notifications.
- Camera mobile pour ajouter documents client, sans exposition publique.
- Tests de composants, hooks, permissions, chargement/erreurs et responsivité.
```

Phase 10 - Intégration et production

Prompt Codex prêt à copier

```
Finalise le MVP production-ready.
- Tests Playwright complets.
- Audit des permissions et fichiers privés.
- manage.py check --deploy.
- compose.prod.yaml et Nginx HTTPS.
- Runbooks sauvegarde/restauration et déploiement LAN + Internet.
- README, MkDocs, ADR et schéma OpenAPI final.
- Rapport des risques résiduels.
Inclure le test concurrent : deux confirmations sur la même ressource, une seule réussit.
```

7. Patterns backend recommandés

7.1 Organisation d'un module

```
apps/bookings/
|-- models.py           # Persistance et contraintes simples
|-- enums.py           # États et types
|-- services/          # Commandes modifiant l'état métier
|-- selectors/         # Requetes de lecture
|-- api/               # Serializers, permissions, views, URLs
|-- events/            # Événements après transaction
|-- tests/             # Models, services, API, concurrence
```

7.2 Service Layer et CQRS léger

Type	Emplacement	Exemple
Commande	services/	confirm_reservation(), register_payment(), close_cash_session()
Lecture	selectors/	get_calendar_view(), get_remaining_balance()
Transport	api/	Validation d'entrée et autorisation HTTP
Événement	events/ / outbox	reservation.confirmed, payment.received

7.3 Verrouillage des opérations critiques

La confirmation de réservation, la sortie d'article et la numérotation des documents doivent être protégées par transaction atomique et verrouillage des lignes. Django propose `select_for_update()` ; PostgreSQL fournit les mécanismes de verrou nécessaires pour les écritures concurrentes. [R4] [R11]

```
@transaction.atomic
def confirm_reservation(*, reservation_id: UUID, actor: User) -> Reservation:
    reservation = Reservation.objects.select_for_update().get(id=reservation_id)
    ensure_contract_is_signed(reservation)
    ensure_deposit_is_received(reservation)

    resources = (
        Resource.objects.select_for_update()
        .filter(id__in=reservation.resource_ids)
        .order_by("id")
    )
    ensure_resources_are_available(resources, reservation.period)
    create_confirmed_allocations(reservation, resources)
    reservation.transition_to(ReservationStatus.CONFIRMED)
    AuditLog.record(actor=actor, action="reservation.confirmed", entity=reservation)
    OutboxEvent.create(event_type="reservation.confirmed",
                       payload={"reservation_id": str(reservation.id)})
    return reservation
```

7.4 Repository Pattern : usage cible

A utiliser pour	A éviter pour
Stockage privé interchangeable de fichiers	Simple CRUD d'article
Calendrier consolidé ou reporting complexe	Liste paginée standard
Intégrations email/export/MVola futures	Modèle utilisateur classique

7.5 Confidentialité documentaire

Document	Accès autorisé
CIN / certificat de résidence	Responsable autorisé, Gérant, Administrateur
NIF / STAT / RCS	Responsable autorisé, Gérant, Administrateur
Justificatifs achats	Gérant, Administrateur, Comptable
Contrat signé / reçus	Rôles habilités du dossier
Exports comptables	Comptable, Gérant, Administrateur

8. Patterns frontend recommandés

8.1 Architecture feature-based

```

features/reservations/
|-- api/
|   |-- reservation.api.ts
|   |-- reservation.keys.ts
|-- components/
|   |-- ReservationForm.tsx
|   |-- ReservationStatusBadge.tsx
|   |-- AvailabilityCalendar.tsx
|-- hooks/
|   |-- useReservation.ts
|   |-- useConfirmReservation.ts
|-- pages/
|   |-- ReservationListPage.tsx
|   |-- ReservationDetailPage.tsx
|-- schemas/
|   |-- reservation.schema.ts
|-- types/
|   |-- reservation.types.ts

```

8.2 Gestion de l'état

Type d'état	Outil	Exemple
Données serveur	TanStack Query	Clients, stock, factures, calendrier
Formulaires	React Hook Form	Création client, encaissement
Validation interface	Zod	Formats et erreurs lisibles
Authentification	Session Django + /auth/me/	Utilisateur courant
Permissions UI	PermissionGate	Masquage menu paie
Temps reel	WebSocket + invalidation query	Stock actualisé

8.3 Règles de sécurité frontend

- Ne jamais considérer le masquage d'un bouton comme une autorisation : l'API doit toujours contrôler l'accès.
- Ne pas conserver de document d'identité, justificatif ou token sensible dans localStorage.
- Utiliser des cookies de session sécurisés pour l'application web.
- Prévoir une interface mobile pour la capture des documents et les contrôles logistiques.

8.4 Limite du mode hors Internet

Clarification

Le fonctionnement sans Internet signifie que l'application reste utilisable sur le LAN tant que le serveur local fonctionne. Un vrai mode mobile hors réseau avec synchronisation différée serait une fonctionnalité supplémentaire.

9. Stratégie de tests

9.1 Pyramide de tests

Niveau	Outils	Objectif
Unitaire backend	pytest	Règles métier isolées.
Services/API	pytest-django + DRF	Transactions, validations et permissions.
Concurrence	pytest + PostgreSQL	Double réservation et numérotation.
Tâches	pytest + Celery	Rappels et idempotence.
Frontend	Vitest + Testing Library	Composants et hooks.
End-to-end	Playwright	Parcours complets navigateur.

9.2 Matrice de tests métier critiques

Domaine	Scénario obligatoire
Réservation	Proforma seul : aucune confirmation.
Réservation	Contrat signé sans acompte : confirmation refusée.
Réservation	Contrat signé + acompte + disponibilité : confirmation réussie.
Concurrence	Deux confirmations simultanées d'une ressource : une seule réussit.
Hahitantsoa/Titan	Ressource confirmée sur un volet indisponible sur l'autre.
Contrat	Modification directe après signature refusée.
Paieement	Chaque paiement valide génère un reçu ; MVola supporté.
Échéances	Rappels J-30 et J-10 générés une seule fois.
Caisse	Paieement cash refusé sans session ouverte.
Caisse	Écart de clôture exige justification.
Stock	Retour intact restaure disponibilité ; casse la retire.
Consommables	Seuil bas déclenche notification/commande.
Sécurité	Document privé inaccessible hors permission.
Audit	Toute validation sensible produit une trace.
Numérotation	Documents concurrents : aucun numéro dupliqué.

9.3 Parcours Playwright MVP

- Connexion et permissions par rôle.
- Création client particulier avec capture CIN et société avec NIF/STAT.
- Agenda visiteur puis conversion en client.
- Hahitantsoa : proforma -> contrat signé -> acompte -> confirmation.
- Titan : location selon la politique de local validée/configurée.
- Paiements J-30 / J-10 et génération de reçus.
- Caisse : ouverture -> encaissement -> comptage -> clôture.
- Bon de sortie -> livraison -> retour intact.

- Retour avec casse -> facture de casse -> caution.
- Alerte consommable -> bon fournisseur.
- Accès interdit aux documents privés.

10. Documentation automatique et gouvernance

10.1 Documents obligatoires

Document	Emplacement	Mise à jour
README	/README.md	Setup, commandes ou déploiement.
Règles métier	/docs/business-rules/	Chaque modification client.
ADR	/docs/adr/	Chaque décision durable.
Architecture	/docs/architecture/	Chaque domaine important.
Schéma OpenAPI	Endpoint/docs versionnés	Chaque changement API.
Runbook déploiement	/docs/runbooks/deployment.md	Infrastructure.
Backup/restore	/docs/runbooks/backup-restore.md	Avant production.
Guide utilisateur	/docs/user-guide/	Avant recette.

10.2 ADR à créer

Référence	Décision
ADR-001	Adoption du monorepo.
ADR-002	Monolithe modulaire plutôt que microservices.
ADR-003	Django 5.2 LTS + React + PostgreSQL.
ADR-004	Confirmation après contrat signé et acompte.
ADR-005	Stock commun Hahitantsoa/Titan.
ADR-006	Politique Titan/local configurable.
ADR-007	Sessions Django plutôt que token navigateur.
ADR-008	Stockage privé des documents sensibles.
ADR-009	Outbox pour notifications critiques.
ADR-010	Docker Compose pour déploiement initial.
ADR-011	OpenAPI avec drf-spectacular.
ADR-012	Accès Internet sécurisé HTTPS.

11. Checklist de validation avant production

11.1 Validation fonctionnelle

Contrôle	Niveau
Périmètre Titan/local confirmé par le client	Obligatoire
Salle par défaut Hahitantsoa validée	Obligatoire
Modèles PDF des documents validés	Obligatoire
Acompte, J-30, J-10 et caution validés	Obligatoire
Rôles et permissions approuvés	Obligatoire
Processus caisse/achats/consommables validé	Obligatoire

11.2 Qualité logicielle

Contrôle	Commande
Backend format	black --check backend
Backend lint	ruff check backend
Backend typing	mypy backend
Backend tests	pytest backend
Migrations	python manage.py makemigrations --check --dry-run
Frontend lint / types / tests	pnpm lint && pnpm typecheck && pnpm test
Frontend build	pnpm build
End-to-end	pnpm playwright test
Conteneurs	docker compose build

11.3 Sécurité et exploitation

Contrôle	Exigence
HTTPS actif	Obligatoire pour accès Internet.
DEBUG=False et secrets externes	A vérifier en production.
PostgreSQL/Redis/Flower	Jamais exposés publiquement.
Documents clients	Accès privé testé par rôle.
Sauvegarde base + fichiers	Automatisée et restauration testée.
Audit	Actif sur contrats, paiements, stocks, caisse.
Django deploy check	Exécution avant go-live.
Formation utilisateurs	Magasinier, accueil, gérant, administrateur, comptable.

12. Ordre concret d'exécution

Étape	Action	Résultat
1	Valider Titan/local et règles finales avec le client.	Ambiguïté levée ou paramètre accepté.
2	Créer nouvelle conversation développement et repository.	Contexte propre et versionné.
3	Installer VS Code, Docker, WSL2 et Codex.	Poste opérationnel.
4	Créer AGENTS.md, PLANS.md et ADR initiaux.	Instructions stables.
5	Exécuter Phase 0 Codex.	Backlog et règles documentés.
6	Exécuter Foundation.	Socle technique, Docker et CI fonctionnels.
7	Identity et permissions.	Sécurité avant métier.
8	Référentiels et fichiers privés.	Données de base disponibles.
9	Réservations et disponibilités.	Cœur transactionnel.
10	Documents et paiements.	Cycle commercial.
11	Caisse, achats, fournisseurs, consommables.	Nouveaux besoins intégrés.
12	Logistique et casse.	Cycle terrain complet.
13	Notifications et frontend final.	Usage opérationnel.
14	Recette, sécurité, déploiement et formation.	Go-live maîtrisé.

13. Organisation des conversations et du contexte de développement

13.1 Recommandation

Créer une nouvelle conversation de développement

La conversation actuelle à remplir son rôle de cadrage et de consolidation client. Pour l'implémentation, un fil dédié réduit les risques de réutiliser des versions obsolètes des règles et permet de travailler avec un jeu documentaire stable.

13.2 Configuration conseillée

Élément	Recommandation
Projet ChatGPT	Conserver le même projet si disponible, afin de regrouper les fichiers et les discussions.
Nouveau chat	Nom suggéré : Développement ERP Hahitantsoa-Titan - Foundation.
Fichiers à joindre	CDC Technique v3.2, Présentation Métier v3.2 et présent Guide de développement.
Premier objectif	Créer le repo, AGENTS.md, PLANS.md et les ADR avant tout code métier.

Élément	Recommandation
Source de vérité	Repository Git et documents versionnés ; jamais un souvenir de conversation seul.
Cette conversation	A garder pour arbitrages métier et retours futurs du client.

13.3 Quand ouvrir un nouveau fil supplémentaire ?

Rester dans le fil de développement pour la phase Foundation et les premiers modules. Ouvrir un fil distinct uniquement lorsqu'un sujet devient autonome et volumineux, par exemple : audit sécurité, stratégie de déploiement, migration de données ou revue d'une fonctionnalité complexe. Dans chaque cas, joindre les documents versionnés pertinents et indiquer le commit ou la branche Git concernée.

13.4 Pourquoi cette organisation est robuste

Risque	Réponse organisationnelle
Confusion entre versions v1, v2 et v3.2	Fil dev démarre uniquement avec les documents consolidés.
Codex produit du code non conforme	AGENTS.md et ADR versionnées dans le repo.
Perte de décisions métier	CDC et business-rules maintenus avec les migrations.
Review difficile	Branches courtes, PR et checkpoints Git.
Contexte trop large	Un epic = une tâche Codex précise ; nouveaux fils seulement si nécessaire.

Annexe A - Premier prompt à exécuter dans Codex

Nous développons l'application Hahitantsoa/Titan ERP décrite dans AGENTS.md.

Ta première mission est une mission de cadrage technique uniquement :

- lis AGENTS.md ;
- analyse la structure vide du repository ;
- crée PLANS.md ;
- propose l'architecture monorepo finale ;
- crée les ADR-001 à ADR-006 ;
- crée les documents de règles métier pour réservations, facturation, caisse et logistique ;
- identifie explicitement les ambiguïtés fonctionnelles restantes, notamment Titan et l'autorisation de louer un local ;
- ne génère pas encore de code Django ou React ;
- ne crée aucune dépendance ;
- termine par une checklist de validation avant de passer à Foundation.

Le livrable doit être lisible par un Tech Lead et utilisable comme contexte stable pour les prochaines tâches Codex.

Annexe B - Références

Ref.	Source / usage
------	----------------

Ref.	Source / usage
R1	OpenAI Help Center - Projets dans ChatGPT : regroupement de fichiers, instructions et conversations pour un travail de longue haleine.
R2	OpenAI Developers - Codex, Custom instructions with AGENTS.md : lecture d'AGENTS.md avant le travail et instructions par projet.
R3	Django Documentation 5.2 - Release notes : version LTS, compatibilité Python et support PostgreSQL.
R4	PostgreSQL Documentation - Explicit Locking : contrôle des accès concurrents aux lignes.
R5	Celery Documentation - Periodic Tasks : planification des tâches récurrentes.
R6	Docker Documentation - Compose : définition et orchestration de services multi-conteneurs.
R7	Django Documentation - Deployment checklist : vérification et sécurisation avant déploiement.
R8	OpenAI Developers - Codex IDE extension : utilisation de Codex dans VS Code.
R9	OpenAI Developers - Codex on Windows : usage natif ou via WSL2.
R10	OpenAI Developers - Codex Quickstart / Best practices : checkpoints Git et validation.
R11	Django Documentation - QuerySet.select_for_update() : verrouillage des lignes depuis l'ORM.
F1	Document B - Présentation Métier v3.1 fourni par le client, 21 mai 2026 : besoins métier consolidés et points à harmoniser.

FIN DU DOCUMENT